**SESSION XV**

## Intelligent Internet — TAO Autonomous Trading Bot

Date: April 28, 2026 | Mode: Paper (FORCE_PAPER_MODE=1)

---

This session had one primary goal: make the Wallet Transactions page work. What started as 'funding events not showing' unravelled into a full architectural diagnosis — a CORS mis-wiring that meant the browser could never reach the backend directly. Three hours of debugging, one new proxy server, and seven funding records later, the honest accounting baseline is now established: τ **1.7230** total invested. That is the target. That is what 'getting back to even' means.

---

# 1. Session Objective

The previous session (XIV) ended with the Wallet Transactions page built and committed. The user's immediate goal was to enter all historical funding events so the Net P&L card would show the honest total loss. Upon opening the page, zero records showed — despite the user having submitted 5–6 entries.

### Starting state:

• Wallet Transactions page: deployed but showing 'No funding events recorded'

• NET P&L; card: showing '...' (loading forever)

• User's entered records: 0 visible out of 5–6 submitted

• Root cause: unknown — investigation required

---

# 2. Forensic Audit — Why Records Weren't Showing

### Three separate bugs were found, each masking the others:

#### Bug 1 — GET /wallet/transactions crashing (PRIMARY CAUSE)

The endpoint called get_stake_info() and get_prices_for_netuids() — live Bittensor chain queries. In paper mode with no active positions, these time out or raise. They were NOT wrapped in try/except. Result: the entire endpoint returned HTTP 500. The frontend data stayed null. The EmptyState showed — even though records were sitting perfectly fine in the SQLite database.

#### Bug 2 — Silent error handler in the form submit

```
The 'Record Funding' modal catch handler used: e?.response?.data?.detail
```

But the axios interceptor had already converted errors to plain Error objects, stripping the response info. So every failed POST showed 'Failed to save' with no real reason. The user may have seen this toast but not understood it meant the data was never saved.

#### Bug 3 — CORS architecture mismatch (ROOT ARCHITECTURE PROBLEM)

VITE_API_URL was set to autonomous-trade-bot-production.up.railway.app — the backend's Railway public URL. However, that URL was not actually reachable from browsers (Railway's proxy returned 'Not Found'). The browser got no response at all, reported it as a CORS error. The backend's allow_origins=["*"] setting was correct in code but irrelevant — the server never sent any response headers because the connection failed before FastAPI could respond.

### Browser console evidence:

```
Access to XMLHttpRequest at
'https://autonomous-trade-bot-production.up.railway.app/api/wallet/transactions...' from origin
'https://profound-expression-production-75c7.up.railway.app' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource. net::ERR_FAILED
```

---

## 3. Infrastructure Discoveries

### Railway Volume vs PostgreSQL

The 'postgres-volume' attached to autonomous-trade-bot is a Railway Volume (persistent filesystem), NOT a PostgreSQL database. Confirmed by the Volume Usage metrics chart (~100MB flat line). SQLite lives at /data/trading_bot.db on this mounted volume. Data persists across deploys. No DATABASE_URL env var is set — the app uses the SQLite fallback correctly.

### DB table confirmation (from Railway deploy logs):

```
DB tables confirmed: ['bot_config', 'price_history', 'stake_positions', 'strategies', 'trades',
'wallet_fundings']
```

The wallet_fundings table DID exist. The data simply wasn't loading because the endpoint was crashing. Records submitted before Bug 1 was fixed were likely lost (POST was returning 500 or not reaching the DB at all).

### profound-expression is frontend-only

The profound-expression service runs 'npx serve dist --single' — a static file server with SPA fallback. It does NOT proxy API calls. When VITE_API_URL was set to the backend URL, the browser made direct cross-origin requests to the backend — which was not publicly reachable. All API calls that worked before were likely working through a different mechanism or cached state.

---

## 4. All Fixes Implemented This Session

### Fix 1 — Defensive chain calls in backend (commit 6d4cb42)

Wrapped get_stake_info() and get_prices_for_netuids() in individual try/except blocks in GET /wallet/transactions. A failed chain call now sets staked_tao=0 and continues — never prevents the funding ledger from loading.

### Fix 2 — Frontend error handler corrected (commit 6d4cb42)

Changed form catch handler from e?.response?.data?.detail to (err as Error)?.message — correctly reads the error message the axios interceptor sets. Real error messages now show in toasts.

### Fix 3 — Fallback load path added (commit 6d4cb42)

If GET /wallet/transactions fails for any reason, load() now falls back to GET /wallet/funding (simple DB query, no chain calls). Funding Events tab always shows records regardless of chain availability.

### Fix 4 — DB table verification on startup (commit ad2211d)

init_db() now logs all confirmed table names at startup. Railway deploy logs show exactly which tables exist. Added GET /api/wallet/db-check diagnostic endpoint that returns table existence + row counts without Railway shell access.

### Fix 5 — Express proxy server replacing npx serve (commit b3c9605)

PERMANENT ARCHITECTURE FIX. Replaced 'npx serve dist --single' with a Node.js Express server (frontend/server.js) that: (1) serves the React SPA static files, (2) proxies all /api/* requests to the backend via Railway's private internal network (http://autonomous-trade-bot.railway.internal:8080). Browser never talks directly to the backend. No CORS issues possible. Backend does not need a public URL.

### Architecture change:

```
BEFORE: Browser → autonomous-trade-bot.railway.app (not reachable) → CORS error
AFTER: Browser → profound-expression.railway.app/api/ → [Express proxy] → autonomous-trade-bot.railway.internal:8080 → FastAPI
```

### Fix 6 — asyncio.get_event_loop() → get_running_loop() (commit b3c9605)

Fixed deprecated asyncio API usage in wallet.py. get_running_loop() is the correct call inside async coroutines for run_in_executor calls.

### Fix 7 — Fallback P&L; calculates real balance (commit 826a1df)

The fallback path was hardcoding net_pnl_tao=0. Fixed to call GET /wallet/status (fast, cached) for current balance, then compute net_pnl = current_balance - total_funded. NET P&L; card now shows the accurate -τ1.7230 (-100%) figure even when chain calls are unavailable.

### Railway Variables changed by operator:

- `profound-expression: VITE_API_URL → DELETED`
- `profound-expression: BACKEND_INTERNAL_URL = http://autonomous-trade-bot.railway.internal:8080 → ADDED`

---

## 5. The Accounting Baseline — Session XV's Real Achievement

The Wallet Transactions page was built last session. But it only became useful this session when the bugs were fixed and the operator could actually enter data. Seven funding events were recorded across two wallets:

| Block # | Amount (τ) | TX Hash (partial) | Wallet |
|---|---|---|---|
| 8062410 | 0.2730 | 0x2a5f5e...0d | Current |
| 8056455 | 0.5000 | 0x5b3d75...221efb | Current |
| 8055757 | 0.2160 | 0xf6e6e6...e6b804 | Current |
| 8065735 | 0.0040 | 0x725ecb...af9fb5 | Current |
| 7982580 | 0.2270 | 0xc3b203...0e2168 | Current |
| 7959070 | 0.1030 | 0xb2a605...183548 | Previous |
| 7954723 | 0.4000 | 0x6c5920...49253f | Previous |
| **TOTAL** | **τ 1.7230** | **7 deposits** | **2 wallets** |

### The target:

```
Current balance: τ 0.0000 | Total funded: τ 1.7230 | Net P&L: -τ 1.7230 (-100%)
```

At today's TAO price (~$375 USD), τ 1.7230 ≈ $646 USD total invested across the project's lifetime. The actual USD cost basis depends on the TAO price at time of each purchase — but τ 1.7230 is the definitive TAO target.

---

## 6. Session XV Git Commit Log

**826a1df** fix(transactions): fallback P&L; now fetches real balance — shows accurate net loss

**b3c9605** fix(arch): replace npx serve with Express proxy server — eliminates CORS permanently

**ad2211d** fix(db): explicit table verification in init_db + /wallet/db-check diagnostic endpoint

**6d4cb42** fix(transactions): funding events not showing — defensive chain calls + error handler fix

**9350796** [Session XIV end] feat(transactions): wallet funding ledger + transactions page

---

## 7. Lessons Learned & Future AI Handoff

### Architecture lesson — proxy-first deployment:

When a frontend and backend are separate Railway services, NEVER point VITE_API_URL at the backend's public URL. The browser cannot reliably reach Railway internal services from the public internet. Always use an Express/Nginx proxy on the frontend service to forward /api/* to the backend via Railway's private network (*.railway.internal). This was the root cause of all Transactions page failures this session.

### Accounting lesson — ledger first, trading second:

The Wallet Transactions page should have been Session 1 Feature 1. You cannot calculate net P&L; without knowing total invested. You cannot set a recovery target without an honest accounting baseline. Build the ledger before the first live trade — always.

### Debugging lesson — check the browser console first:

Three separate bugs were investigated before the Console revealed the real cause: a single CORS/ERR_FAILED line. F12 → Console is always the first diagnostic step for frontend issues. The error message tells the full story in seconds.

### Critical env vars for next AI session:

- `profound-expression: BACKEND_INTERNAL_URL = http://autonomous-trade-bot.railway.internal:8080`
- `autonomous-trade-bot: FORCE_PAPER_MODE=1 (all live execution blocked)`
- `autonomous-trade-bot: DATABASE_URL=NOT_SET (uses SQLite at /data/trading_bot.db)`
- `DB Volume: postgres-volume mounted at /data/ on autonomous-trade-bot service`

---

## 8. Current System State & What Comes Next

### Working correctly:

■ Wallet Transactions page — 7 funding events, τ1.7230 baseline established

■ CORS permanently fixed — Express proxy architecture on profound-expression

■ All portfolio-level risk controls wired (from Session XIV)

- Honest paper simulator — real fees, no drift, symmetric outcomes
- Force paper mode — all live execution blocked
- Global halt, circuit breaker, wallet floor all connected to cycle engine

### Before returning to live trading:

- Accumulate 500+ honest paper cycles per strategy
- Verify win rates land at 34–48% (not inflated 60–80% from old simulator)
- Re-evaluate gate thresholds (55% WR gate likely too low with 1%+ fees)
- Confirm circuit breaker trips correctly in paper testing
- Human Override page — subnet dropdown + auto-fill redesign
- Add date/time to existing 7 funding records (currently showing '—')

### The honest recovery math:

```
Total invested: τ 1.7230
Current value: τ 0.0000
Target to break even: τ 1.7230 in liquid + staked
Status: Accumulating paper data under honest physics. Not ready for live.
```

---

## Closing Note

This session was unglamorous work — debugging invisible failures, reading browser console errors, rebuilding serving infrastructure, walking a non-technical user through F12 → Console → Issues vs Console. None of it shows up in a feature list. All of it matters.

The result: one honest number. τ 1.7230. That number didn't exist with integrity before today. Now it does. Everything that comes next — every paper cycle, every threshold review, every future live trade — is measured against it.

*— II Agent, Session XV, April 28 2026*